

AWS SELF LEARNING & HANDS -ON IMPLEMENTATION ACTIVITY

AWS Lambda • Amazon API Gateway • AWS KMS

Complete Concepts, Architecture, Configuration, Testing & Troubleshooting

Primary Practical Region: us-east-1

Core architecture

Client → API Gateway → Lambda → KMS

API Gateway = API entry point

Lambda = application logic

KMS = managed cryptographic key management

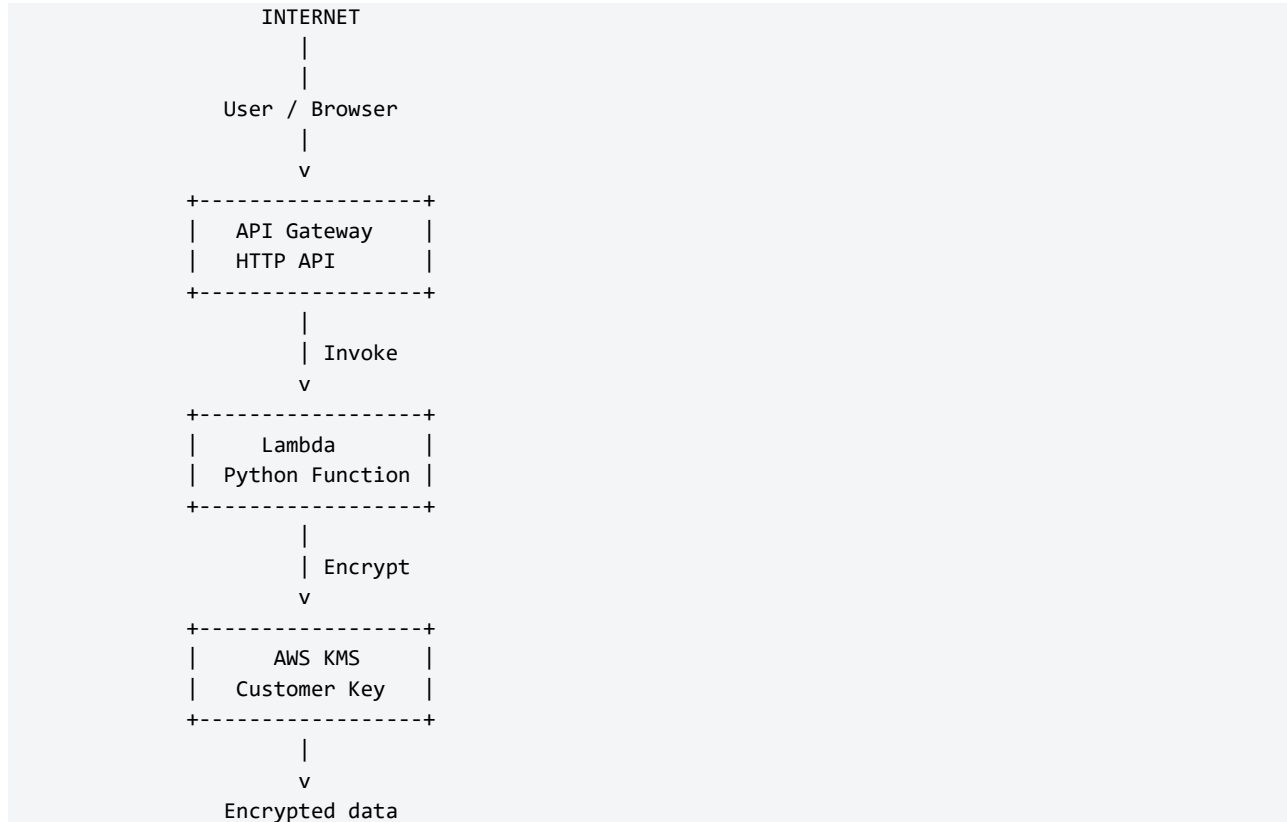
Contents

1. Big Picture — How These Three Work Together

Imagine you have an application where a user sends:

```
POST /encrypt
{
  "message": "Hello AWS"
}
```

The architecture can be:



The responsibilities are different:

Service	Main responsibility
API Gateway	Receives HTTP requests
Lambda	Executes application logic
KMS	Protects encryption keys / performs cryptographic operations
IAM	Controls who can access what
CloudWatch	Logs and monitors Lambda/API activity

AWS describes Lambda as a serverless compute service where functions execute in response to events. A Lambda function uses an **execution role** to obtain permissions for AWS resources. ([AWS Documentation](#))

API Gateway can expose an HTTP endpoint and invoke Lambda when a request arrives. ([AWS Documentation](#))

KMS provides managed cryptographic keys and cryptographic operations, with access controlled through key policies, IAM policies and grants. ([AWS Documentation](#))

PART 1 — AWS Lambda

2. What is AWS Lambda?

AWS Lambda is AWS's **serverless compute service**.

It allows you to run code without creating or managing:

- EC2 servers
- operating systems
- application servers
- web servers
- server patches
- server scaling configuration

You upload your code and AWS executes it when something invokes the function.

For example:

```
User uploads image
↓
S3 event
↓
Lambda
↓
Resize image
↓
Store resized image
```

Or:

```
HTTP request
↓
API Gateway
↓
Lambda
↓
Database
↓
Response
```

3. Lambda — Simple Real-World Example

Suppose you want:

```
GET /hello
```

The Lambda function could contain:

```
def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "body": "Hello from AWS Lambda!"
    }
```

When the function executes:

```

API Gateway
  |
  v
Lambda
  |
  v
Python code
  |
  v
HTTP response

```

4. Important Lambda Terminology

You should know these terms for interviews.

Function

The actual Lambda application.

Example:

```
hello-world-function
```

Runtime

The environment used to execute your code.

Examples include:

```

Python
Node.js
Java
.NET
Go
Ruby

```

For our lab, we'll use:

```
Python
```

Handler

The function that Lambda calls.

Example:

```
def lambda_handler(event, context):
```

Here:

```
lambda_handler
```

is the handler function.

Event

The input received by Lambda.

For example:

```
{
  "name": "Sunith"
}
```

Lambda receives this as:

```
event
```

Context

Lambda provides execution information through:

```
context
```

It can contain information such as:

- function name
- request ID
- remaining execution time
- memory information

5. Lambda Execution Role

This is extremely important.

Lambda itself needs permission to access AWS services.

For example:

```
Lambda
|
+----> CloudWatch Logs
|
+----> S3
|
+----> DynamoDB
|
+----> KMS
```

The permissions are provided through an **IAM execution role**.

AWS documentation states that every Lambda function has an execution role, and Lambda assumes this role when the function executes. ([AWS Documentation](#))

For our project:

```
Lambda Execution Role
|
+---- CloudWatch Logs
|
+---- KMS Encrypt/Decrypt
```

6. Lambda Memory

You can configure Lambda memory.

For example:

```
128 MB
256 MB
512 MB
1024 MB
...
```

Important:

More memory also gives the function more CPU capacity.

Therefore, increasing memory can sometimes make a function execute faster.

7. Lambda Timeout

Timeout determines how long Lambda is allowed to execute.

Example:

```
Timeout = 10 seconds
```

If the function does not finish within that period:

```
Lambda
|
|----- 10 seconds -----|
|                                     X
|                               Timeout
```

You need to choose the timeout according to the workload.

8. Lambda Environment Variables

You can store configuration such as:

```
DATABASE_HOST
DATABASE_NAME
ENVIRONMENT
KMS_KEY_ID
```

Example:

```
KMS_KEY_ID = alias/my-application-key
```

But **do not treat ordinary Lambda environment variables as a secure password vault.**

For secrets, use services such as:

- AWS Secrets Manager
- AWS Systems Manager Parameter Store

and control access through IAM.

This is generally faster.

11. Lambda Layers

A Lambda Layer allows you to package reusable code or dependencies separately from your function code.

For example:

```
Lambda Function
|
+--- application code
|
+--- Layer
|
+--- common libraries
```

Useful when multiple functions share dependencies.

12. Lambda Versions and Aliases

You can publish versions:

```
Function
|
+-- Version 1
|
+-- Version 2
|
+-- Version 3
```

Then create aliases:

```
production → Version 3
testing    → Version 4
```

This is useful for deployments.

13. Lambda Monitoring

Lambda integrates with CloudWatch.

You should monitor:

- Invocations
- Errors
- Duration
- Throttles
- Concurrent executions
- Logs

Example:

```

Lambda
|
+----> CloudWatch Metrics
|
+----> CloudWatch Logs

```

PART 2 — AWS KMS

14. What is AWS KMS?

AWS Key Management Service is AWS's managed service for creating and controlling cryptographic keys.

Its main purpose is:

Protect data by controlling and managing encryption keys.

AWS KMS provides cryptographic operations through a managed service backed by AWS's HSM infrastructure. KMS keys are not exported to you as plaintext key material. ([AWS Documentation](#))

15. Why Do We Need KMS?

Imagine you have:

```

Customer information
Passwords
Financial information
API secrets
Database information

```

You don't want sensitive information stored as:

```
Hello AWS
```

You want:

```
8f92a7e1c9...
```

or another encrypted representation.

Encryption requires keys.

Conceptually:

```

Plaintext
|
| Encryption Key
v
Ciphertext

```

To get the original information back:

```

Ciphertext
|
| Decryption Key
v
Plaintext

```

KMS helps you manage those encryption keys securely.

16. Very Important — KMS Is Not Just "An Encryption Service"

This is a common interview mistake.

KMS is primarily a **key management and cryptographic service**.

Think:

```

KMS
|
+--- Create keys
|
+--- Manage keys
|
+--- Control access
|
+--- Encrypt/decrypt data
|
+--- Generate data keys
|
+--- Key rotation
|
+--- Auditing through CloudTrail

```

17. KMS Key Types

AWS KMS documentation distinguishes:

AWS owned keys

Managed entirely by AWS.

You generally don't manage their policies or lifecycle.

AWS managed keys

Keys created and managed by AWS services for your account.

Customer managed keys

Keys you create and control.

For learning and hands-on work, we'll create:

```
Customer managed symmetric KMS key
```

AWS documents the differences between AWS owned, AWS managed and customer managed keys. ([AWS Documentation](#))

18. Symmetric vs Asymmetric KMS Keys

Symmetric

Same cryptographic key is used for encryption/decryption operations.

Conceptually:

```
Key
|
+---- Encrypt
|
+---- Decrypt
```

This is the common choice for encrypting application data.

Asymmetric

Uses:

```
Public key
Private key
```

Conceptually:

```
Public Key
  ↓
Encryption

Private Key
  ↓
Decryption
```

Asymmetric keys can also be used for signing/verification depending on key specification.

19. Envelope Encryption

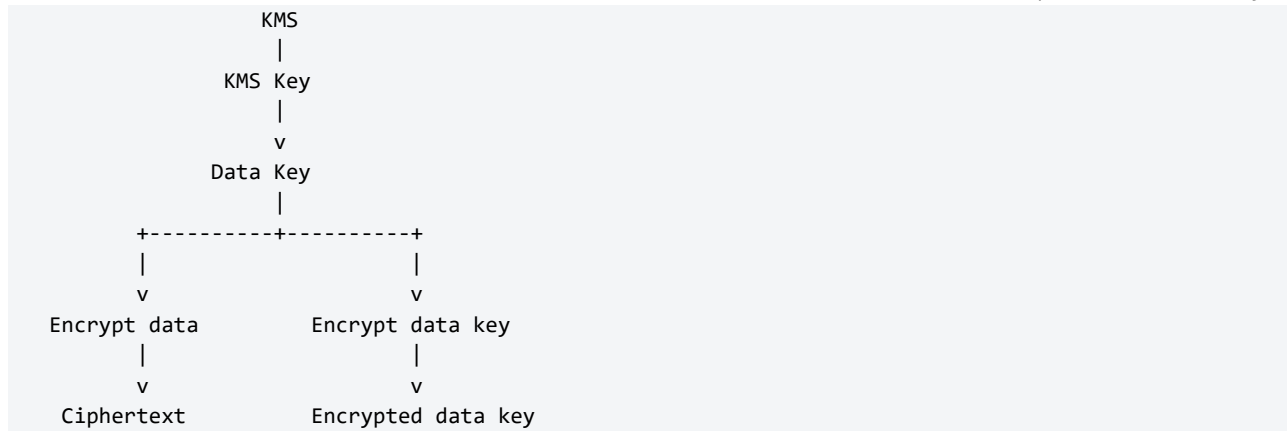
This is one of the most important KMS concepts.

Suppose you have a large file:

```
1 GB file
```

You don't necessarily want KMS to directly encrypt every byte of the file.

Instead, a common architecture is:



This is called **envelope encryption**.

AWS describes envelope encryption as encrypting plaintext data using a data key and then encrypting that data key under another key. ([AWS Documentation](#))

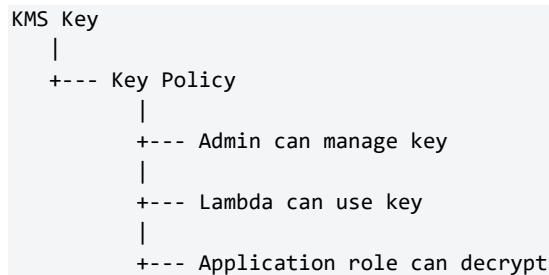
This is heavily used by AWS services.

20. KMS Key Policy

This is **extremely important**.

A KMS key has a key policy.

Example concept:



Every KMS key has exactly one key policy. KMS authorization can involve:

```

Key Policy
IAM Policy
Grant
  
```

AWS specifically notes that a KMS key policy is required and that IAM policies need the key policy to permit IAM-based access. ([AWS Documentation](#))

21. KMS Grants

A grant is another mechanism for allowing a principal or AWS service to use a KMS key.

They're particularly useful for delegated or temporary access.

AWS services commonly use grants when working with customer-managed KMS keys. ([AWS Documentation](#))

For your first lab, you don't need to manually create grants.

PART 3 — API GATEWAY

22. What is Amazon API Gateway?

Amazon API Gateway is a managed AWS service used to create, publish, secure, monitor and manage APIs.

Think:

```

Client
|
| HTTP Request
v
API Gateway
|
| Route request
v
Backend
  
```

The backend could be:

```

Lambda
HTTP application
AWS service
  
```

AWS documents API Gateway REST APIs as collections of resources and methods that can integrate with Lambda, HTTP endpoints, or AWS services. ([AWS Documentation](#))

23. Why API Gateway?

Suppose you have Lambda:

```

Lambda
  
```

How does a browser call it?

API Gateway gives you:

```

https://xxxxx.execute-api.us-east-1.amazonaws.com/...
  
```

Then:

```

Browser
|
| HTTPS
v
API Gateway
|
v
Lambda
  
```

24. API Gateway Types You Should Know

The two major API Gateway API types you'll encounter are:

HTTP API

Designed to provide a simpler, lower-cost API option with fewer features.

REST API

Provides more advanced API management capabilities.

AWS describes REST APIs as having more features, while HTTP APIs are designed with a more minimal feature set. ([AWS Documentation](#))

There is also:

WebSocket API

Used for persistent two-way communication.

Example:

```
Chat application
Live notifications
Real-time communication
```

25. API Gateway Important Concepts

You should understand:

```
API
|
+--- Route / Resource
|
+--- Method
|
+--- Integration
|
+--- Stage
|
+--- Deployment
|
+--- Authorization
|
+--- Throttling
|
+--- CORS
|
+--- Monitoring
```

26. HTTP Methods

Common methods:

```
GET
POST
PUT
PATCH
DELETE
```

For example:

```
GET /users
POST /users
GET /users/123
PUT /users/123
DELETE /users/123
```

27. API Gateway + Lambda Integration

Architecture:

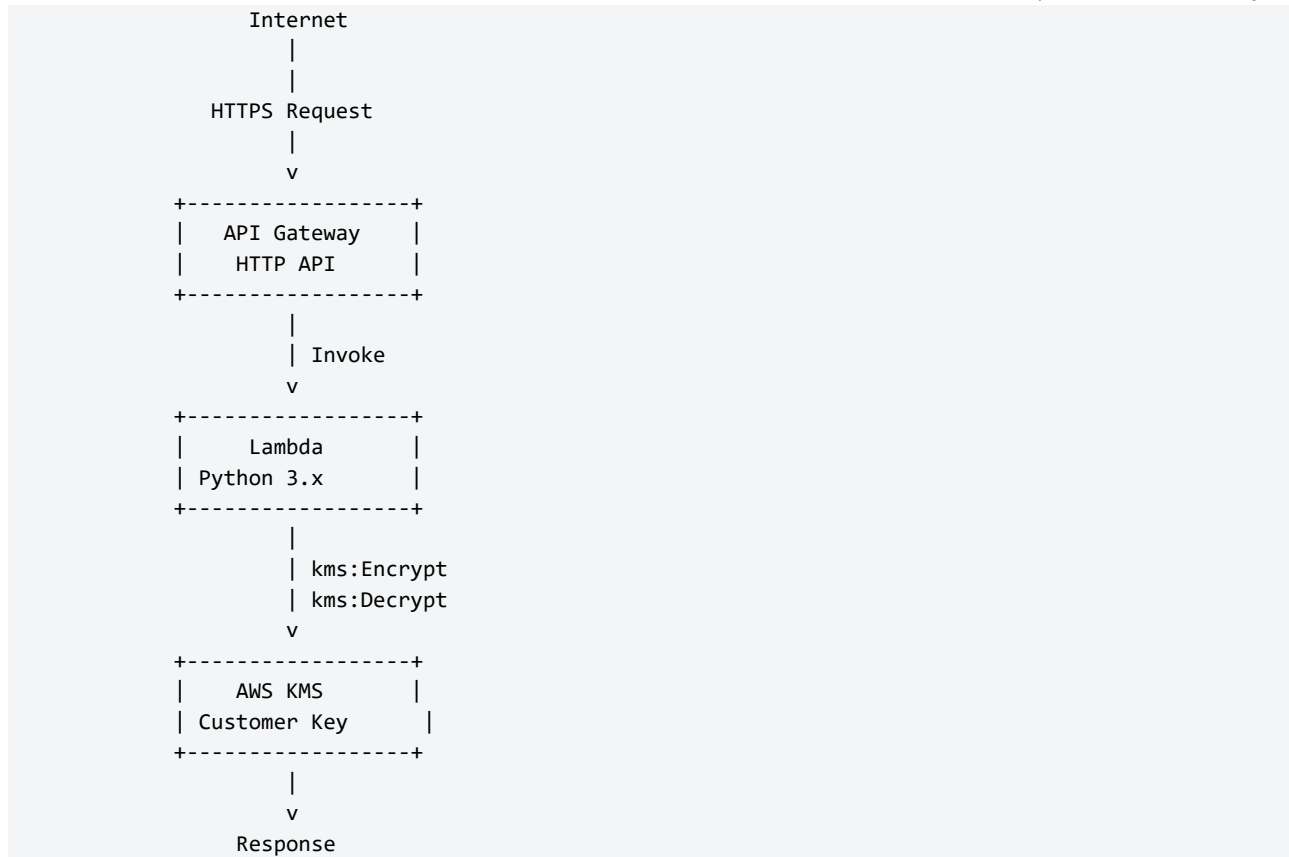
```
Client
|
| GET /hello
v
API Gateway
|
| Lambda invocation
v
Lambda
|
| response
v
API Gateway
|
v
Client
```

For REST APIs, API Gateway can use Lambda proxy integration, where the request is passed to Lambda and Lambda's response is returned through API Gateway. ([AWS Documentation](#))

PART 4 — COMPLETE HANDS-ON PROJECT

Now let's build all three together.

We're going to create:



We'll implement two endpoints:

```
GET /hello
POST /encrypt
```

28. Lab Prerequisites

Use your AWS region:

```
us-east-1
```

You'll need permissions to create:

```
Lambda
IAM role/policies
KMS key
API Gateway
CloudWatch resources
```

Important

KMS customer-managed keys can incur charges, so for a Free Tier practice account, **delete the test key after completing the lab if you don't need it**. Check the current AWS pricing page before creating resources for cost-sensitive work.

29. STEP 1 — Create KMS Key

Open:

AWS Console → KMS

Then:

```
Key Management Service
  ↓
Customer managed keys
  ↓
Create key
```

Key configuration

Choose:

```
Key type:
Symmetric
```

For key usage:

```
Encrypt and decrypt
```

Click:

```
Next
```

30. Add Key Alias

Enter:

```
Alias:
alias/lambda-api-key
```

Description:

```
KMS key for Lambda API Gateway training project
```

Tags are optional.

Continue.

31. Configure Key Administrators

You will see:

```
Key administrators
```

Select the IAM user/role you use to administer the AWS account.

For a personal lab, select your administrative identity.

Do not blindly give every application role key administration permissions.

This is an important security principle:

Administrator
≠
Application

32. Configure Key Users

For now, you can initially leave application use to the appropriate role configuration and then explicitly authorize the Lambda execution role.

The important architecture is:

```

Key administrator
  |
  | manage key
  v
KMS Key

Lambda execution role
  |
  | use key
  v
KMS Key

```

Click:

Next

Review.

Click:

Finish

33. Record the KMS Key ID

Open your newly created key.

You'll see something like:

```

Key ARN:
arn:aws:kms:us-east-1:123456789012:key/xxxxxxx...

```

And:

```

Key ID:
xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx

```

Record it.

Also note:

```

Alias:
alias/lambda-api-key

```

34. STEP 2 — Create Lambda Function

Go to:

```
AWS Console  
→ Lambda  
→ Functions  
→ Create function
```

Choose:

```
Author from scratch
```

Function name:

```
lambda-kms-api-demo
```

Runtime:

```
Python
```

Choose the current supported Python runtime shown in your console.

Architecture:

```
x86_64
```

For the initial lab, use the default execution role option that creates a basic Lambda execution role.

Click:

```
Create function
```

35. STEP 3 — Write Lambda Code

Go to:

```
Lambda  
→ Functions  
→ lambda-kms-api-demo  
→ Code
```

Replace the sample code with:

```

import json
import boto3
import base64
import os

kms = boto3.client("kms")

KMS_KEY_ID = os.environ["KMS_KEY_ID"]

def lambda_handler(event, context):

    print("Received event:", json.dumps(event))

    path = event.get("rawPath", "")

    # GET /hello
    if path == "/hello":

        return {
            "statusCode": 200,
            "headers": {
                "Content-Type": "application/json"
            },
            "body": json.dumps({
                "message": "Hello from AWS Lambda!",
                "service": "Lambda",
                "region": os.environ.get("AWS_REGION")
            })
        }

    # POST /encrypt
    if path == "/encrypt":

        body = event.get("body", "")

        if not body:
            return {
                "statusCode": 400,
                "body": json.dumps({
                    "error": "Request body is required"
                })
            }

        try:

            request_data = json.loads(body)

            message = request_data.get("message")

            if not message:
                return {
                    "statusCode": 400,
                    "body": json.dumps({
                        "error": "message field is required"
                    })
                }

            response = kms.encrypt(
                KeyId=KMS_KEY_ID,

```

```

        Plaintext=message.encode("utf-8")
    )

    ciphertext = response["CiphertextBlob"]

    encoded_ciphertext = base64.b64encode(
        ciphertext
    ).decode("utf-8")

    return {
        "statusCode": 200,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps({
            "message": "Encryption successful",
            "ciphertext": encoded_ciphertext
        })
    }

except Exception as e:

    print("Encryption error:", str(e))

    return {
        "statusCode": 500,
        "body": json.dumps({
            "error": "Encryption failed",
            "details": str(e)
        })
    }

return {
    "statusCode": 404,
    "body": json.dumps({
        "error": "Route not found"
    })
}

```

Click:

Deploy

36. IMPORTANT — Why Are We Using Environment Variables?

Notice:

```
KMS_KEY_ID = os.environ["KMS_KEY_ID"]
```

We don't hard-code:

```
KMS_KEY_ID = "xxxxxxxx..."
```

Instead we configure:

```

Lambda
→ Configuration
→ Environment variables

```

Add:

Key:
KMS_KEY_ID

Value:

your KMS key ARN

For example:

arn:aws:kms:us-east-1:123456789012:key/xxxxxxx

Click:

Save

37. STEP 4 — Give Lambda Permission to Use KMS

This is where IAM becomes important.

Go:

Lambda
→ Your function
→ Configuration
→ Permissions

Find:

Execution role

Click the role name.

This opens:

IAM
→ Role

38. Add KMS Permission

Choose:

Add permissions
→ Create inline policy

Choose:

KMS

Select permissions needed by our function:

kms:Encrypt

If you later add decryption functionality:

kms:Decrypt

You can also need:

kms:DescribeKey

depending on your implementation.

39. Restrict the Permission to Your Key

Do **not** unnecessarily use:

```
Resource: *
```

Prefer your specific KMS key.

For example:

```
arn:aws:kms:us-east-1:123456789012:key/xxxxxxxx...
```

This follows the principle of least privilege.

Policy conceptually:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "kms:Encrypt"
      ],
      "Resource": "YOUR-KMS-KEY-ARN"
    }
  ]
}
```

Name it:

```
LambdaKMSAccessPolicy
```

Create policy.

40. VERY IMPORTANT — KMS Key Policy

If Lambda gets:

```
kms:Encrypt
```

through IAM but the KMS key policy doesn't permit the relevant account/identity model, the call may still fail.

This is one of the most common KMS troubleshooting areas.

Remember:

```
IAM policy
+
KMS key policy
+
possibly grants
```

control access.

AWS explicitly documents that key policies are central to KMS authorization. ([AWS Documentation](#))

41. STEP 5 — Test Lambda Directly

Go back:

```
Lambda
→ Function
→ Test
```

Create a test event.

For example:

```
{
  "rawPath": "/hello"
}
```

Run:

```
Test
```

Expected result:

```
{
  "statusCode": 200,
  "headers": {
    "Content-Type": "application/json"
  },
  "body": "{\"message\": \"Hello from AWS Lambda!\", ...}"
}
```

42. Test Encryption

Create another test event:

```
{
  "rawPath": "/encrypt",
  "body": "{\"message\": \"Hello AWS KMS\"}"
}
```

Run.

Expected:

```
{
  "statusCode": 200,
  "body": "{\"message\": \"Encryption successful\", \"ciphertext\": \"...\"}"
}
```

The ciphertext should not look like:

```
Hello AWS KMS
```

It will be encoded encrypted binary data.

43. STEP 6 — Check CloudWatch Logs

Go:

```
Lambda
→ Monitor
→ View CloudWatch logs
```

You'll find a log group similar to:

```
/aws/lambda/lambda-kms-api-demo
```

Inside you'll see Lambda invocation logs.

This is extremely useful for troubleshooting.

PART 5 — API GATEWAY CONFIGURATION

44. STEP 7 — Create HTTP API

Go to:

```
AWS Console
→ API Gateway
```

Choose:

```
Create API
```

Under:

```
HTTP API
```

choose:

```
Build
```

HTTP APIs can integrate directly with Lambda. AWS's HTTP API documentation describes creating an API with an integration, route and stage. ([AWS Documentation](#))

45. Add Lambda Integration

Choose:

```
Add integration
```

Select:

```
Lambda
```

Region:

```
us-east-1
```

Lambda function:

```
lambda-kms-api-demo
```

API name:

```
lambda-kms-http-api
```

Continue.

46. Create `/hello` Route

Create route:

```
GET /hello
```

Integration:

```
Lambda  
→ lambda-kms-api-demo
```

Conceptually:

```
GET /hello  
  |  
  v  
API Gateway  
  |  
  v  
Lambda  
  |  
  v  
Hello response
```

47. Create `/encrypt` Route

Add another route:

```
POST /encrypt
```

Integration:

```
Lambda  
→ lambda-kms-api-demo
```

Now:

```
GET /hello  
POST /encrypt
```

both invoke the same Lambda.

48. Stage

HTTP API can use a default stage with automatic deployment.

You may see:

```
$default
```

For a learning environment, this is convenient.

Conceptually:

```
API
|
+--- $default
```

49. Obtain API Endpoint

API Gateway will provide an endpoint similar to:

```
https://xxxxxxxxx.execute-api.us-east-1.amazonaws.com
```

Your actual URL will be different.

Do not copy mine.

50. Test GET API

Open:

```
https://YOUR_API_ID.execute-api.us-east-1.amazonaws.com/hello
```

Expected:

```
{
  "message": "Hello from AWS Lambda!",
  "service": "Lambda",
  "region": "us-east-1"
}
```

You have now completed:

```
Browser
  ↓
API Gateway
  ↓
Lambda
  ↓
Response
```

51. Test POST Encryption

Using curl:

```
curl -X POST \
  "https://YOUR_API_ID.execute-api.us-east-1.amazonaws.com/encrypt" \
  -H "Content-Type: application/json" \
  -d '{"message":"Hello AWS KMS"}'
```

Expected response:

```
{
  "message": "Encryption successful",
  "ciphertext": "..."
```

Now the complete flow is:

```

curl
|
| POST /encrypt
v
API Gateway
|
| Lambda invocation
v
Lambda
|
| kms:Encrypt
v
KMS
|
| Ciphertext
v
Lambda
|
v
API Gateway
|
v
curl

```

PART 6 — ADD DECRYPTION

Now we can make the project much more useful.

Add:

```
POST /decrypt
```

The client sends the encrypted ciphertext.

Lambda calls:

```
kms:Decrypt
```

and returns the plaintext.

52. Update Lambda Code

Add this logic before the final 404 response:

```

if path == "/decrypt":

    body = event.get("body", "")

    if not body:
        return {
            "statusCode": 400,
            "body": json.dumps({
                "error": "Request body is required"
            })
        }

    try:

        request_data = json.loads(body)

        encoded_ciphertext = request_data.get("ciphertext")

        if not encoded_ciphertext:
            return {
                "statusCode": 400,
                "body": json.dumps({
                    "error": "ciphertext field is required"
                })
            }

        ciphertext = base64.b64decode(
            encoded_ciphertext
        )

        response = kms.decrypt(
            CiphertextBlob=ciphertext
        )

        plaintext = response["Plaintext"].decode("utf-8")

        return {
            "statusCode": 200,
            "headers": {
                "Content-Type": "application/json"
            },
            "body": json.dumps({
                "message": "Decryption successful",
                "plaintext": plaintext
            })
        }

    except Exception as e:

        print("Decryption error:", str(e))

        return {
            "statusCode": 500,
            "body": json.dumps({
                "error": "Decryption failed",
                "details": str(e)
            })
        }

```

Deploy.

53. Add IAM Permission

Your Lambda execution role currently has:

```
kms:Encrypt
```

Add:

```
kms:Decrypt
```

So:

```
"Action": [  
    "kms:Encrypt",  
    "kms:Decrypt"  
]
```

54. Add API Route

API Gateway:

```
API Gateway  
→ APIs  
→ lambda-kms-http-api  
→ Routes  
→ Create
```

Create:

```
POST /decrypt
```

Integration:

```
lambda-kms-api-demo
```

55. Complete Encryption Test

First:

```
curl -X POST \  
  "https://YOUR_API_ID.execute-api.us-east-1.amazonaws.com/encrypt" \  
  -H "Content-Type: application/json" \  
  -d '{"message":"Hello AWS KMS"}'
```

You'll receive:

```
{  
  "message": "Encryption successful",  
  "ciphertext": "AQICAH..."  
}
```

Copy:

```
ciphertext
```

56. Decrypt

Then:

```
curl -X POST \
  "https://YOUR_API_ID.execute-api.us-east-1.amazonaws.com/decrypt" \
  -H "Content-Type: application/json" \
  -d '{"ciphertext":"PASTE_CIPHERTEXT_HERE"}'
```

Expected:

```
{
  "message": "Decryption successful",
  "plaintext": "Hello AWS KMS"
}
```

Congratulations — you've created a complete:

```
API Gateway
  ↓
Lambda
  ↓
KMS
  ↓
Encryption / Decryption
```

architecture.

PART 7 — SECURITY CONFIGURATION

The basic lab works, but production architecture needs additional security.

57. API Authentication

You generally don't want every API to be publicly callable.

API Gateway supports different authorization mechanisms depending on API type and configuration.

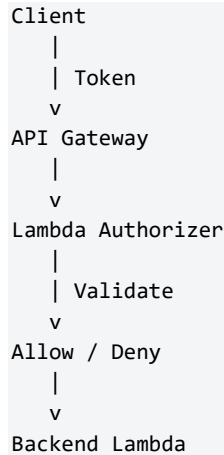
Examples include:

```
IAM authorization
JWT authorization
Lambda authorizer
Cognito
```

For HTTP APIs, JWT/OAuth 2.0 authorization is supported. ([AWS Documentation](#))

58. Lambda Authorizer

A Lambda authorizer works like:



AWS documents Lambda authorizers as a mechanism where API Gateway invokes an authorizer function to authenticate/authorize the caller before invoking the API integration. ([AWS Documentation](#))

59. CORS

Suppose:

```

Website
https://example.com
  
```

calls:

```

API Gateway
  
```

Browser security can require CORS configuration.

Configure:

```

API Gateway
→ CORS
  
```

Specify allowed origins.

For learning:

```

*
  
```

may be convenient, but **don't blindly use `\*` in a production API** when you know which frontend origins should be allowed.

60. API Throttling

Suppose somebody sends:

```

1,000,000 requests
  
```

to your API.

API Gateway supports throttling.

Conceptually:

```

Client
|
| 1000 requests/sec
v
API Gateway
|
| permitted traffic
v
Lambda

```

Excess requests can receive:

```

HTTP 429
Too Many Requests

```

API Gateway uses token-bucket throttling, and configured limits should be viewed as targets rather than guaranteed hard ceilings. ([AWS Documentation](#))

61. Lambda Concurrency Protection

API Gateway throttling and Lambda concurrency are related but different.

Example:

```

API Gateway
|
| 500 requests
v
Lambda
|
| concurrency limit
v
Controlled execution

```

You can configure Lambda reserved concurrency to prevent one function from consuming all available concurrency.

PART 8 — MONITORING

For a production-style setup:

```

API Gateway
|
+---- CloudWatch metrics
|
Lambda
|
+---- CloudWatch Logs
|
KMS
|
+---- CloudTrail

```

62. Lambda Metrics

Monitor:

```

Invocations
Errors
Duration
Throttles
ConcurrentExecutions

```

63. API Gateway Metrics

Monitor:

```

Requests
4XX errors
5XX errors
Latency
Integration latency

```

If:

```
4XX ↑
```

investigate client/API configuration.

If:

```
5XX ↑
```

investigate backend/API integration problems.

64. KMS Monitoring

KMS API activity can be audited through CloudTrail.

For example:

```

Lambda
|
| Encrypt
v
KMS
|
v
CloudTrail

```

You can investigate:

```

Who called KMS?
Which key?
Which operation?
When?
From where?

```

PART 9 — TROUBLESHOOTING

This part is especially important for your practical work.

65. Problem: Lambda Gives AccessDeniedException

Example:

```
AccessDeniedException
not authorized to perform kms:Encrypt
```

Check:

```
Lambda
↓
Execution role
↓
IAM policy
↓
KMS key policy
```

Make sure the Lambda execution role has:

```
kms:Encrypt
```

and the KMS authorization configuration permits the role/account.

66. Problem: `kms:Decrypt` Access Denied

If encryption works:

```
Encrypt = SUCCESS
```

but decryption fails:

```
Decrypt = AccessDenied
```

check that your Lambda role has:

```
kms:Decrypt
```

Don't assume:

```
kms:Encrypt
```

automatically means:

```
kms:Decrypt
```

Permissions are action-specific.

67. Problem: Lambda Can't Write Logs

If you see CloudWatch permission errors:

check Lambda execution role.

It should have permissions for CloudWatch Logs, typically through the Lambda basic execution role.

The Lambda console's standard execution role setup provides basic log permissions. ([AWS Documentation](#))

68. Problem: API Gateway Returns 500

Check:

```
API Gateway
  ↓
Integration
  ↓
Lambda
  ↓
CloudWatch Logs
```

Open Lambda logs.

Look for:

```
ERROR
Exception
Traceback
```

69. Problem: API Gateway Returns 404

Check:

```
URL
Route
HTTP method
Stage
Deployment
```

For example:

You created:

```
POST /encrypt
```

but you're sending:

```
GET /encrypt
```

That won't match the route.

70. Problem: API Gateway Returns 403

Check:

```
Authorization
IAM permissions
JWT
Lambda authorizer
API resource policy
```

If using a Lambda authorizer, check its returned authorization result as well. AWS notes that an authorization denial can result in a 403 ACCESS_DENIED response. ([AWS Documentation](#))

71. Problem: API Gateway Returns 429

Likely related to throttling.

Check:

```
API Gateway throttling
Lambda concurrency
Account quotas
Client request rate
```

A 429 Too Many Requests response can occur when API Gateway throttling thresholds are exceeded. ([AWS Documentation](#))

72. Problem: Lambda Timeout

Example:

```
Task timed out
```

Check:

```
Lambda
→ Configuration
→ General configuration
→ Timeout
```

But don't simply increase the timeout.

First identify **why** it's slow.

For example:

```
Lambda
↓
Database
↓
Network problem
↓
Timeout
```

Increasing timeout might hide the actual problem.

73. Problem: Lambda KMS Timeout

Check:

```
Lambda
  ↓
KMS
```

If Lambda is running inside a VPC and needs to access KMS, verify the network design.

A Lambda function attached to a VPC does not automatically have internet access.

Depending on your architecture, you may need appropriate VPC connectivity such as:

```
NAT Gateway
```

or a suitable:

```
VPC endpoint / PrivateLink configuration
```

This is an important distinction from a Lambda function that isn't attached to your VPC.

74. Problem: Wrong KMS Region

KMS keys are regional resources.

For example:

```
Lambda → us-east-1
KMS key → us-west-2
```

You need to understand the regional relationship and use the appropriate key/service endpoint.

A KMS key policy applies to the KMS key in its own Region. ([AWS Documentation](#))

For your lab:

```
Lambda      us-east-1
API Gateway  us-east-1
KMS          us-east-1
```

Keep everything in one region.

75. Problem: API Gateway Doesn't Invoke Lambda

Check:

```
API Gateway
  → Integration
```

Verify the Lambda function is correct.

For REST APIs, API Gateway needs permission to invoke the Lambda function. AWS documents this as part of Lambda integration setup. ([AWS Documentation](#))

PART 10 — INTERVIEW EXPLANATION

If your interviewer asks:

"What is Lambda?"

You can say:

AWS Lambda is a serverless compute service that lets us execute application code without managing servers. Lambda functions can be triggered by AWS services or HTTP requests through API Gateway. Lambda automatically manages the underlying execution infrastructure and scales based on invocations.

76. "What is API Gateway?"

Say:

Amazon API Gateway is a managed service used to create, publish, secure, monitor and manage APIs. It acts as the entry point for clients and can route HTTP requests to services such as Lambda, HTTP backends and other AWS services.

77. "What is KMS?"

Say:

AWS Key Management Service is a managed cryptographic key management service. It allows us to create and control encryption keys, manage access to those keys, and perform cryptographic operations such as encryption and decryption. KMS is also integrated with many AWS services for encryption at rest.

78. "Why Use Lambda With API Gateway?"

Answer:

```

API Gateway
  |
  | HTTP
  v
Lambda
  
```

API Gateway handles:

```

HTTP
Routing
Authorization
Throttling
CORS
API management
  
```

Lambda handles:

Application logic
Processing
AWS service interaction

79. "Why Use KMS With Lambda?"

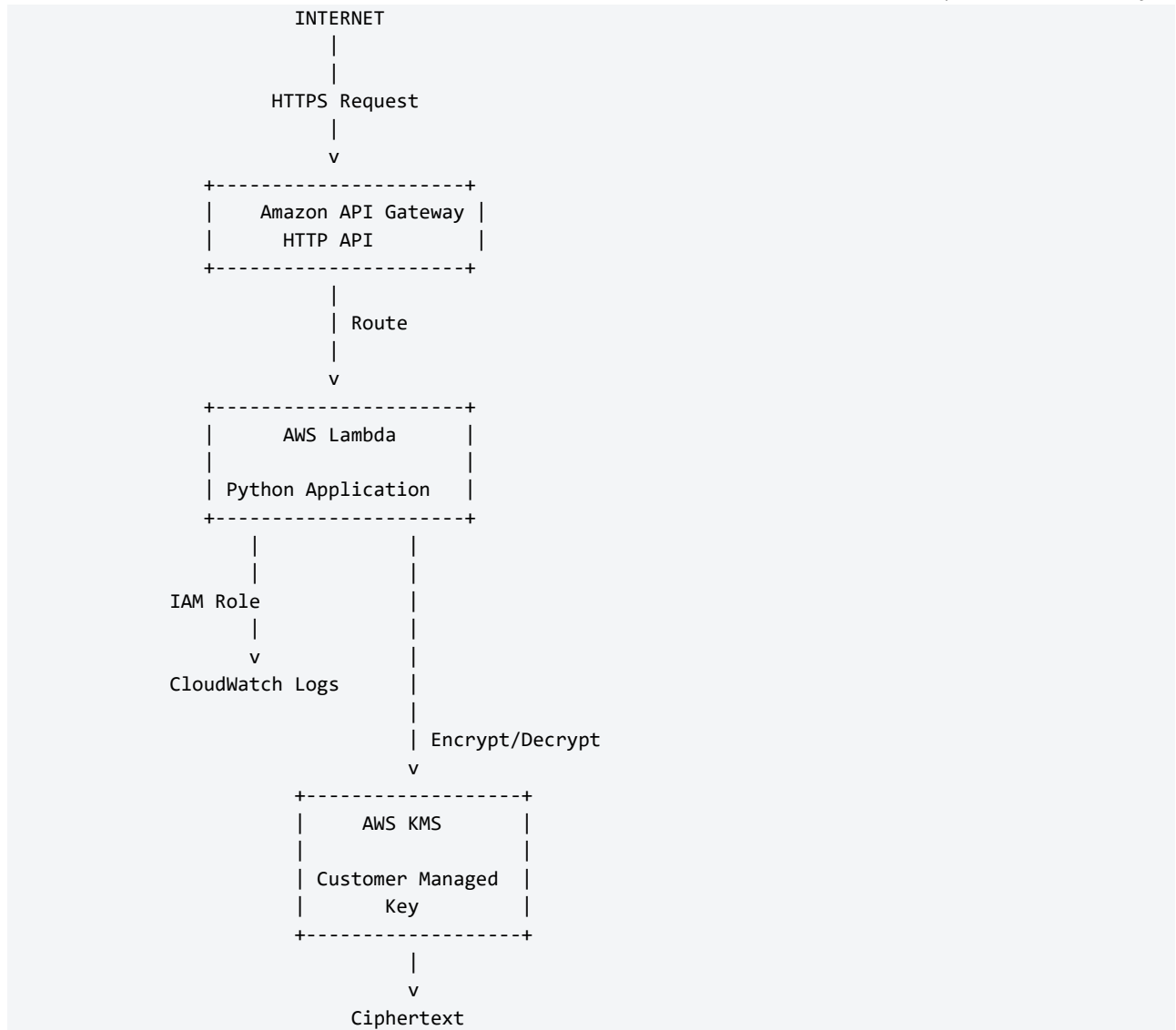
Because Lambda may need to:

Encrypt sensitive data
Decrypt protected data
Generate data keys
Access encrypted AWS resources

KMS provides centralized key management and authorization.

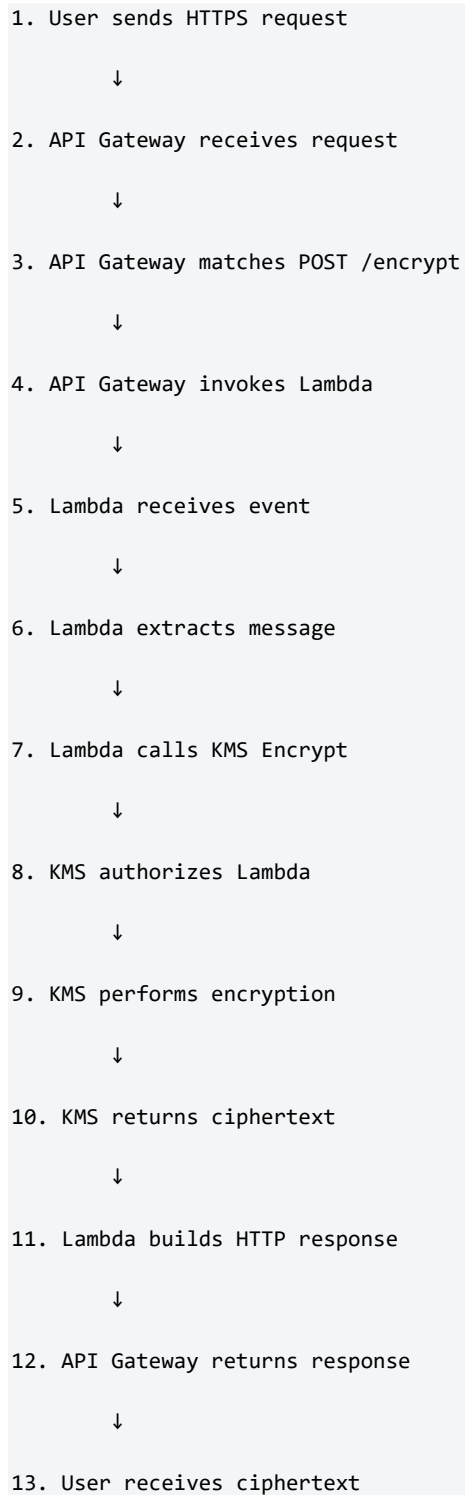
80. Complete Architecture You Should Remember

This is the architecture I recommend you draw in your AWS notes/draw\..io:



81. End-to-End Request Flow

For encryption:



82. Security Flow

The security model is:

```

User
|
| HTTPS
v
API Gateway
|
| Authorization
v
Lambda
|
| IAM execution role
v
KMS
|
| Key policy
v
Customer managed KMS key

```

This demonstrates a very important AWS principle:

Authentication/authorization and encryption are different responsibilities.

API Gateway controls access to your API.

IAM controls AWS permissions.

KMS controls cryptographic keys and operations.

Lambda executes your application logic.

83. The Most Important Differences

Feature	Lambda	API Gateway	KMS
Main purpose	Run code	Expose/manage APIs	Manage encryption keys
Server management	No	No	No
HTTP endpoint	Not its main role	Yes	No
Executes code	Yes	No	No
Encryption	Can call KMS	Can integrate with security mechanisms	Yes
IAM	Execution role	API authorization/integration permissions	Key policies/IAM/grants
CloudWatch	Yes	Yes	AWS service activity can be audited with CloudTrail
Scaling	Automatic	Managed	Managed
Typical use	Backend logic	API frontend	Encryption/key management

84. The Three Services in One Sentence

Remember this:

API Gateway receives the request, Lambda executes the business logic, and KMS protects sensitive data through managed cryptographic keys.

That sentence is enough to establish the relationship in an interview.

85. Recommended Practical Learning Sequence

Since you're practicing AWS hands-on, don't try to memorize these services independently.

Do them in this order:

LAB 1 — Lambda only

Create:

```
Lambda
→ Python function
→ Test event
→ CloudWatch logs
```

Understand:

```
Function
Runtime
Handler
Event
Context
Execution role
Timeout
Memory
Environment variables
Logs
```

LAB 2 — KMS only

Create:

```
Customer managed KMS key
```

Learn:

```
Key ID
ARN
Alias
Key policy
IAM policy
Encrypt
Decrypt
Key rotation
CloudTrail
```

LAB 3 — API Gateway + Lambda

Build:

```
Client
↓
API Gateway
↓
Lambda
```

Create:

```
GET /hello
POST /hello
```

Learn:

```
Routes
Methods
Integration
Stage
Deployment
CORS
Authorization
Throttling
```

LAB 4 — Lambda + KMS

Build:

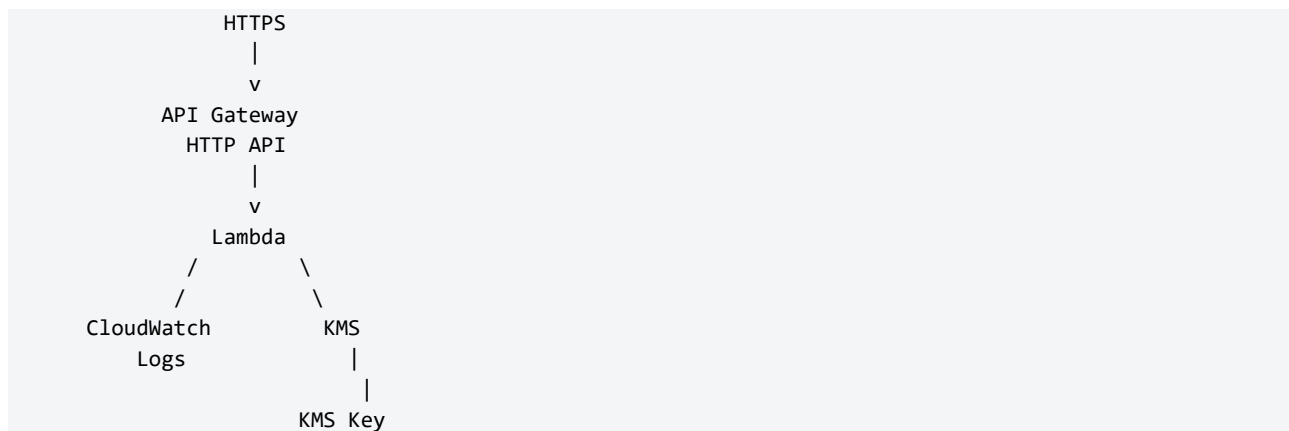
```
Lambda
↓
IAM
↓
KMS
```

Test:

```
Encrypt
Decrypt
```

LAB 5 — Complete Project

Finally:



This final lab is the one you should document in your AWS practical notebook.

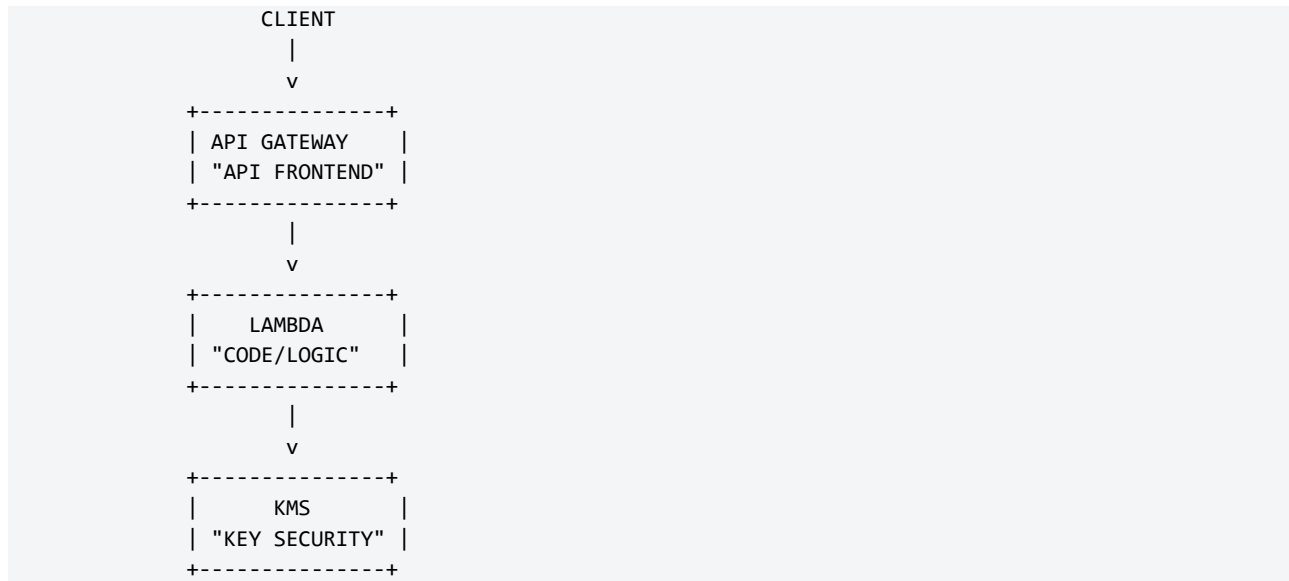
86. AWS Documentation to Keep for Your Notes

These are the official AWS documentation areas I used for the explanation:

- [AWS Lambda — How Lambda works](#) — Lambda execution model, roles and fundamentals. ([AWS Documentation](#))
- [AWS KMS — Concepts](#) — KMS fundamentals and cryptographic concepts. ([AWS Documentation](#))
- [AWS KMS — Key policies](#) — Critical for understanding KMS permissions. ([AWS Documentation](#))
- [AWS KMS — Envelope encryption](#) — How data keys and KMS keys work together. ([AWS Documentation](#))
- [API Gateway — HTTP APIs](#) — HTTP API architecture and features. ([AWS Documentation](#))
- [API Gateway — Lambda integrations](#) — Lambda/API Gateway integration. ([AWS Documentation](#))
- [AWS tutorial — Lambda with API Gateway](#) — Complete AWS example of API Gateway invoking Lambda. ([AWS Documentation](#))

The most important practical takeaway

Don't think of these as three unrelated AWS services:



API Gateway = door

Lambda = worker

KMS = secure key vault/cryptographic service

For your AWS practical, the **next best exercise is to perform the complete project above from a completely fresh setup in `us-east-1`, using the AWS Console, with every screen/navigation choice and every value to enter documented as `Objective → Purpose → Prerequisites → Step-by-step configuration → Testing → Troubleshooting → Cleanup → Conclusion`**. That will turn these concepts into an actual working AWS lab rather than just theory.

Quick Revision — 3 Services in One Sentence

API Gateway receives the request → Lambda executes the business logic → KMS protects sensitive data through managed cryptographic keys.

API Gateway = door | Lambda = worker | KMS = secure key / cryptographic service

Hands on Implementation:

Lab 1 Document

Objective

To create and execute an AWS Lambda function and understand its basic execution model, runtime, handler, event, execution role, and CloudWatch logging.

Purpose

Lambda provides serverless compute, allowing application code to run without managing EC2 servers or operating systems.

Steps

Document:

1. Open Lambda.
2. Create function.
3. Select Python runtime.
4. Configure function name.
5. Create execution role.
6. Add Python code.
7. Deploy.
8. Create test event.
9. Execute test.
10. Verify response.
11. Open CloudWatch logs.
12. Verify execution logs.

Result

Lambda Function ↓ Python Code ↓ Execution ↓ CloudWatch Logs

Implementation :

Successfully updated the function "lambda-kms-api-demo".

Executing function: succeeded (logs 2)

Details

Response

```
{
  "statusCode": 200,
  "body": "Hello from AWS Lambda!"
}
```

Summary

Code SHA-256	Execution time
9Uxv6pQ5N8208j1Q+4s7cmFg8DfDL2mY3H4E2U+	1 second ago
Function version	Request ID
\$LATEST	41dd3bc<e895-4b05-9706-6d63107392ac
Duration	Billed duration
1.79 ms	106 ms
Resources configured	Max memory used
128 MB	39 MB
Init duration	
101.52 ms	

Log output

The area below shows the last 4 KB of the execution log. [Click here](#) to view the corresponding CloudWatch log group.

```
START RequestID: 438613c6-e895-4b05-9706-6d63107392ac VERSION: $LATEST
Lambda Function executed
END RequestID: 438613c6-e895-4b05-9706-6d63107392ac
REPORT RequestID: 438613c6-e895-4b05-9706-6d63107392ac Duration: 1.79 ms    Billed Duration: 186 ms Memory Size: 128 MB    Max Memory Used: 39 MB    Init Duration: 101.52 ms
```

Test event

CloudWatch Logs Live Tail Save Test

The screenshot displays the AWS CloudWatch console interface. At the top, the browser address bar shows the URL: `us-east-1.console.aws.amazon.com/cloudwatch/home?region=us-east-1#logsV2:log-groups/log-group/$252faws$252flambda-kms-api-demo/log-events...`. The console header includes the AWS logo, navigation icons, a search bar, and the text "[Alt+F] Ask Amazon Q".

The main content area is titled "CloudWatch" and shows the breadcrumb path: `CloudWatch > Log management > /aws/lambda/lambda-kms-api-demo > 2026/09/21/[SLATEST]pf96a775c7d941d6b36020f2eeaf711`. Below this, the "Log events" section is active. It contains a search bar with the placeholder text "Filter events - press enter to search" and a set of filters: "Clear", "1m", "30m", "1h", "12h", "Custom", and "U".

The log events are displayed in a table with two columns: "Timestamp" and "Message". The first event is a message stating "No older events at this moment. Retry". The subsequent events are log entries from the Lambda function, including "START RequestId: 4160c8c-8995-4085-9796-6d03d87392ac" and "END RequestId: 4160c8c-8995-4085-9796-6d03d87392ac". The interface also shows a sidebar with navigation options like "Ingestion", "Dashboards", "Alarms", "AI Operations", "GenAI Observability", and "Application Signals (APM)".

Successfully created and tested an AWS Lambda function and verified its execution through CloudWatch Logs.

Lab 2 AWS KMS Only

Objective

To create a customer-managed AWS KMS key and understand encryption, decryption, key identifiers, key policies and access control.

Purpose

KMS centrally manages cryptographic keys and provides cryptographic operations.

Steps

1. Open KMS.
2. Create customer-managed key.
3. Select symmetric key.
4. Select Encrypt and decrypt.
5. Configure alias.
6. Configure administrator.
7. Create key.
8. Record Key ID.
9. Record ARN.
10. Record alias.
11. Review key policy.
12. Perform encryption.
13. Perform decryption.
14. Verify original plaintext.

Implementation:

The screenshot displays the AWS KMS console interface for a specific key. The breadcrumb navigation shows the path: KMS > Customer-managed keys > Key ID: 5484b071-524b-439e-ba5f-acbc0cf99402. The left sidebar contains navigation links for Key Management Service (KMS), AWS managed keys, Customer-managed keys, Custom key stores, AWS CloudHSM key stores, and External key stores.

The main content area is titled with the key ID: 5484b071-524b-439e-ba5f-acbc0cf99402. It features a 'General configuration' section with the following details:

- Alias:** alias/lambda-api-key
- Status:** Enabled
- Creation date:** Sep 21, 2026 17:57 GMT+5:30
- ARN:** arn:aws:kms:us-east-1:567487919764:key/5484b071-524b-439e-ba5f-acbc0cf99402
- Description:** KMS key for Lambda API Gateway training project
- Last used:** No record since key creation
- Current key material ID:** 7a5370c5c4466035b315732a5506f1c7b6399dce580beca09543e488ee0ff
- Regionality:** Single region

Below the general configuration, there are tabs for 'Key policy', 'Cryptographic configuration', 'Tags', 'Key material and rotations', and 'Aliases'. The 'Key policy' tab is active, showing a 'Key policy' section with a 'Switch to policy view' button. Underneath, the 'Key administrators (0)' section is visible, indicating no administrators are currently configured. The 'Key deletion' section shows a checkbox for 'Allow key administrators to delete this key'. At the bottom, the 'Key users (0)' section is also shown with no users configured.

```

Samala Sneha@DESKTOP-KOF17A5 MINGW64 /
$ aws --version
aws-cli/2.34.4 Python/3.13.11 Windows/11 exe/AMD64

Samala Sneha@DESKTOP-KOF17A5 MINGW64 /
$ aws configure
AWS Access Key ID [*****DNV7]: AKIAYIIHK02KCOLGBEVP
AWS Secret Access Key [*****5A4f]: zWwAwEcve6oXEgN3TCTI5ila4imoF+ODub1//K9n
Default region name [us-east-1]: us-east-1
Default output format [json]: json

Samala Sneha@DESKTOP-KOF17A5 MINGW64 /
$ aws sts get-caller-identity
{
  "UserId": "567487919764",
  "Account": "567487919764",
  "Arn": "arn:aws:iam::567487919764:root"
}

Samala Sneha@DESKTOP-KOF17A5 MINGW64 /
$ aws kms describe-key --key-id alias/lambda-api-key

Samala Sneha@DESKTOP-KOF17A5 MINGW64 /
$ aws kms describe-key \
> --key-id 5484b071-524b-439e-ba5f-acbc0cf99402 \
> --region us-east-1
{
  "KeyMetadata": {
    "AWSAccountId": "567487919764",
    "KeyId": "5484b071-524b-439e-ba5f-acbc0cf99402",
    "Arn": "arn:aws:kms:us-east-1:567487919764:key/5484b071-524b-439e-ba5f-acbc0cf99402",
    "CreationDate": "2026-09-21T17:57:23.902000+05:30",
    "Enabled": true,
    "Description": "KMS key for Lambda API Gateway training project",
    "KeyUsage": "ENCRYPT_DECRYPT",
    "KeyState": "Enabled",
    "Origin": "AWS_KMS",
    "KeyManager": "CUSTOMER",
    "CustomerMasterKeySpec": "SYMMETRIC_DEFAULT",
    "KeySpec": "SYMMETRIC_DEFAULT",
    "EncryptionAlgorithms": [
      "SYMMETRIC_DEFAULT"
    ],
    "MultiRegion": false,
    "CurrentKeyMaterialId": "7a5370c5c4466035b315732a9506f1c7b6399cbe58c0beca0b9543e488eee9ff"
  }
}

Samala Sneha@DESKTOP-KOF17A5 MINGW64 /
$ echo -n "Hello AWS KMS" > plaintext.txt
Samala Sneha@DESKTOP-KOF17A5 MINGW64 /
$ cat plaintext.txt
Hello AWS KMS
Samala Sneha@DESKTOP-KOF17A5 MINGW64 /
$ aws kms encrypt \
> --key-id 5484b071-524b-439e-ba5f-acbc0cf99402 \
> --plaintext fileb://plaintext.txt \
> --region us-east-1 \
> --output text \
>
AQZCAHhU3DfKXZgNMcVcyqBvHtJmclJAvsOLtUPk1o7pWZAAOU5S1XrCgierwmqAAAAzBp8gkqH1G9w08BwagDBAaEAMFUGCSqG51B3QeHATAeBg1ghkgBZQMEAS4wEQpw1U3gB6kzw052RvAgTQcgaqTHua+CJoqd4PgpcA8Yjtbew8hue99JctfVYS/G380WnpF11988
07487919764:key/5484b071-524b-439e-ba5f-acbc0cf99402

```

Conclusion

Successfully created and tested a customer-managed KMS key for encryption and decryption.

Lab 3 Create HTTP API

LAB 3 Documentation

Objective

To expose an AWS Lambda function through Amazon API Gateway using an HTTP API.

Purpose

API Gateway provides the HTTP/API entry point while Lambda performs the backend application logic.

Steps

1. Open API Gateway.
2. Create HTTP API.
3. Create Lambda integration.
4. Select Lambda function.
5. Create /hello route.
6. Configure GET method.
7. Configure stage.
8. Obtain API endpoint.
9. Test API.
10. Verify Lambda invocation.
11. Check CloudWatch logs.

Implementation:

lambda-kms-http-api

Stages: -

Deploy

API details

API ID

vdi5ome7pg

Description

-

ARN

arn:aws:apigateway:us-east-1::apis/vdi5ome7pg

Protocol

HTTP

IP address type

IPv4

Created

2026-09-21

Default endpoint

Enabled

https://vdi5ome7pg.execute-api.us-east-1.amazonaws.com

Edit

Stages for lambda-kms-http-api (1)

Find resources

Stage name	Invoke URL	Attached deployment	Auto deploy	Last updated
\$default	https://vdi5ome7pg.execute-api.us-east-1.amazonaws.com	i1lqc9	enabled	2026-09-21

Tags (0)

Find resources

Key

Value

No tags

No tags associated with the resource.

Manage tags

```

Samalā Sneha@DESKTOP-KOF17A5 MINGW64 ~/kms-lab
$ curl -i https://vdi5ome7pg.execute-api.us-east-1.amazonaws.com/hello
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           % Dload  % Upload   Total   Spent    Left     Speed
100    24    100    24    0    0    19      0  0:00:01  0:00:01 --:--:-- 19HTTP/2 200
date: Mon, 21 Sep 2026 16:08:52 GMT
content-type: text/plain; charset=utf-8
content-length: 24
apigw-requestid: EDP3JixkoAMEbBg=

Hello from Sunith Kumar!

```

```

Samalā Sneha@DESKTOP-KOF17A5 MINGW64 ~/kms-lab
$ |

```

Name: enter name (optional) Save Share Generate

<https://vdi5ome7pg.execute-api.us-east-1.amazonaws.com/hello> GET Send

[Params](#) [Body](#) [Auth](#) [Headers 4](#) [Raw 5](#)

Query Params

Key	Value
key	value
key	value
key	value

[Body 1](#) [Headers 5](#) [Raw 8](#) [Timings](#) 200 (OK) 80 ms 0.02 kb

Body

```

Hello from Sunith Kumar!

```

Conclusion

Successfully integrated API Gateway with Lambda and exposed Lambda functionality through an HTTPS endpoint.

4 Lambda environment variable

Objective

The objective of this lab is to integrate **AWS Lambda** with **AWS Key Management Service (AWS KMS)** to perform encryption securely.

In this lab, a Lambda function is configured to:

- Retrieve the KMS key ARN from an environment variable.
- Use its IAM execution role to access the KMS key.
- Encrypt a message using the KMS Encrypt operation.
- Convert the resulting ciphertext to Base64 so it can be returned through an HTTP-style response.
- Test the Lambda function directly using a Lambda test event.

The lab demonstrates the relationship between **Lambda, IAM permissions, and KMS**

Purpose

The purpose of this lab is to understand how AWS Lambda can securely use a customer-managed KMS key without storing the key information directly in the application code.

Step 1: Open the Lambda Function

AWS Console → Lambda → Functions → lambda-kms-api-demo

Step 2: Open Environment Variables

Configuration → Environment variables

Click Edit and add the following variable:

Key: KMS_KEY_ID

Value: arn:aws:kms:us-east-1:567487919764:key/5484b071-524b-439e-ba5f-acbc0cf99402

Why use an environment variable?

Using an environment variable keeps configuration information outside the Python source code.

Instead of writing: KMS_KEY_ID = "arn:aws:kms:..."

the Lambda function can retrieve the value using:

```
import os
KMS_KEY_ID = os.environ["KMS_KEY_ID"]
```

Task 4.2 — Configure IAM Permission

Open the Lambda Execution Role

Lambda → Configuration → Permissions → Execution role

Add an Inline Policy

Add permissions → Create inline policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "kms:Encrypt"
      ],
      "Resource": "arn:aws:kms:us-east-1:567487919764:key/5484b071-524b-439e-ba5f-acbc0cf99402"
    }
  ]
}
```

Add Encryption Code

Update the code:

The Lambda function is configured to retrieve the KMS key ARN from the environment variable.

```
import json
import boto3
import base64
import os

kms = boto3.client("kms", region_name="us-east-1")

KMS_KEY_ID = os.environ["KMS_KEY_ID"]

def lambda_handler(event, context):

    body = json.loads(event.get("body", "{}"))
    message = body.get("message", "")

    response = kms.encrypt(
        KeyId=KMS_KEY_ID,
        Plaintext=message.encode("utf-8")
    )

    ciphertext = base64.b64encode(
        response["CiphertextBlob"]
    ).decode("utf-8")

    return {
        "statusCode": 200,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps({
            "message": "Encryption successful",
```

```

    "ciphertext": ciphertext
  })
}

```

Test Lambda Directly

Create a Test Event :Test → Create new event

Use the following JSON:

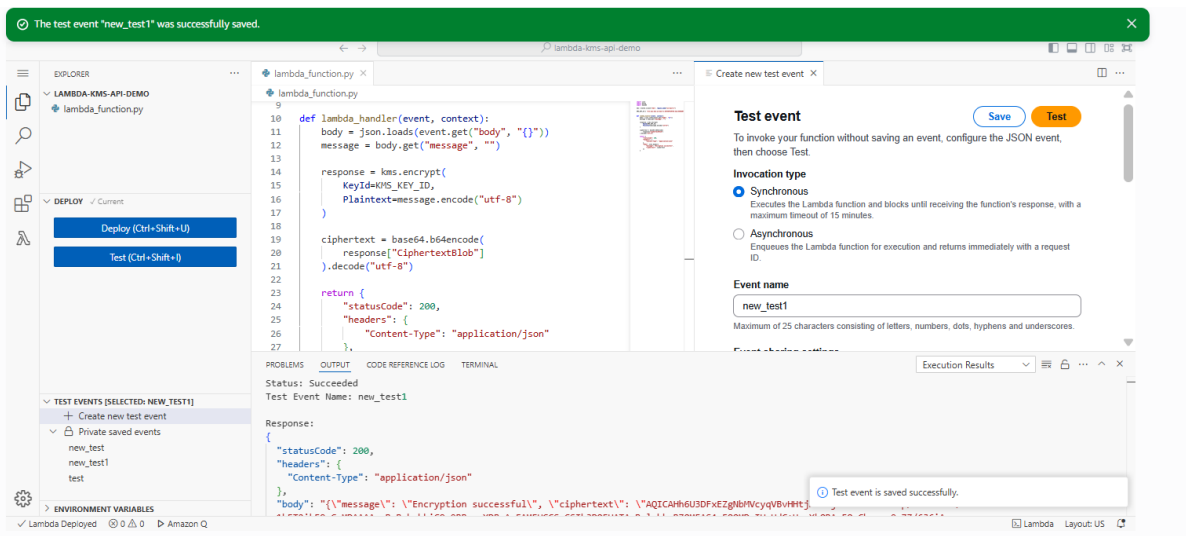
```

{
  "rawPath": "/encrypt",
  "body": "{\"message\":\"Hello AWS KMS\"}"
}

```

Run the Test

Implementation:



Conclusion:

In this lab, AWS Lambda was successfully integrated with AWS Key Management Service (KMS).

The Lambda function used an environment variable to obtain the KMS key ARN and used its IAM execution role to obtain permission to perform the kms:Encrypt operation.

Lab 5 COMPLETE AWS LAMBDA + API GATEWAY + KMS PROJECT

LAB 5 — COMPLETE AWS LAMBDA + API GATEWAY + KMS PROJECT

Objective

The objective of this lab is to combine Amazon API Gateway, AWS Lambda, IAM, AWS KMS, CloudWatch Logs, and CloudTrail into one complete serverless application.

The final application provides three HTTP endpoints:

- GET /hello — Returns a basic Lambda response.
- POST /encrypt — Encrypts a message using the customer-managed KMS key.
- POST /decrypt — Decrypts previously encrypted ciphertext using the same KMS key.

Purpose:

The purpose of this lab is to demonstrate how multiple AWS services can work together to create a secure serverless API.

The project demonstrates:

- Creating HTTP endpoints using API Gateway.
- Connecting API Gateway to Lambda.
- Using Lambda to process HTTP requests.
- Using an IAM execution role to authorize KMS operations.
- Encrypting sensitive messages using AWS KMS.
- Decrypting ciphertext using AWS KMS.
- Returning JSON responses to API clients.
- Observing Lambda execution through CloudWatch Logs.
- Auditing KMS API activity through CloudTrail.

Prepared the lambda function:

Use the following Lambda code for the complete project:

```
import json
import boto3
import base64
import os
```

```
kms = boto3.client("kms", region_name="us-east-1")
```

```
KMS_KEY_ID = os.environ["KMS_KEY_ID"]
```

```
def lambda_handler(event, context):
```

```
path = event.get("rawPath", "")
method = event.get("requestContext", {}).get("http", {}).get("method", "")
```

```
# GET /hello
```

```
if path == "/hello" and method == "GET":
```

```
    return {
        "statusCode": 200,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps({
            "message": "Hello from AWS Lambda!",
            "service": "Lambda",
            "region": "us-east-1"
        })
    }
}
```

```
# POST /encrypt
```

```
elif path == "/encrypt" and method == "POST":
```

```
    body = json.loads(event.get("body", "{}"))
    message = body.get("message", "")
```

```
    response = kms.encrypt(
        KeyId=KMS_KEY_ID,
        Plaintext=message.encode("utf-8")
    )
```

```
    ciphertext = base64.b64encode(
        response["CiphertextBlob"]
    ).decode("utf-8")
```

```
    return {
        "statusCode": 200,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps({
            "message": "Encryption successful",
            "ciphertext": ciphertext
        })
    }
}
```

```
# POST /decrypt
```

```
elif path == "/decrypt" and method == "POST":
```

```

body = json.loads(event.get("body", "{}"))
ciphertext = body.get("ciphertext", "")

encrypted_data = base64.b64decode(ciphertext)

response = kms.decrypt(
    KeyId=KMS_KEY_ID,
    CiphertextBlob=encrypted_data
)

plaintext = response["Plaintext"].decode("utf-8")

return {
    "statusCode": 200,
    "headers": {
        "Content-Type": "application/json"
    },
    "body": json.dumps({
        "message": "Decryption successful",
        "plaintext": plaintext
    })
}

return {
    "statusCode": 404,
    "headers": {
        "Content-Type": "application/json"
    },
    "body": json.dumps({
        "message": "Route not found"
    })
}

```

Create the API Gateway HTTP API

AWS Console → API Gateway

Create API → HTTP API

Connect API Gateway to Lambda

Select your Lambda function: lambda-kms-api-demo

Configure the /hello Route

Create the route: GET /hello

Connect the route to: lambda-kms-api-demo

Test the /hello Endpoint : curl -I https://YOUR_API_ID.execute-api.us-east-1.amazonaws.com/hello
https://YOUR_API_ID.execute-api.us-east-1.amazonaws.com/hello

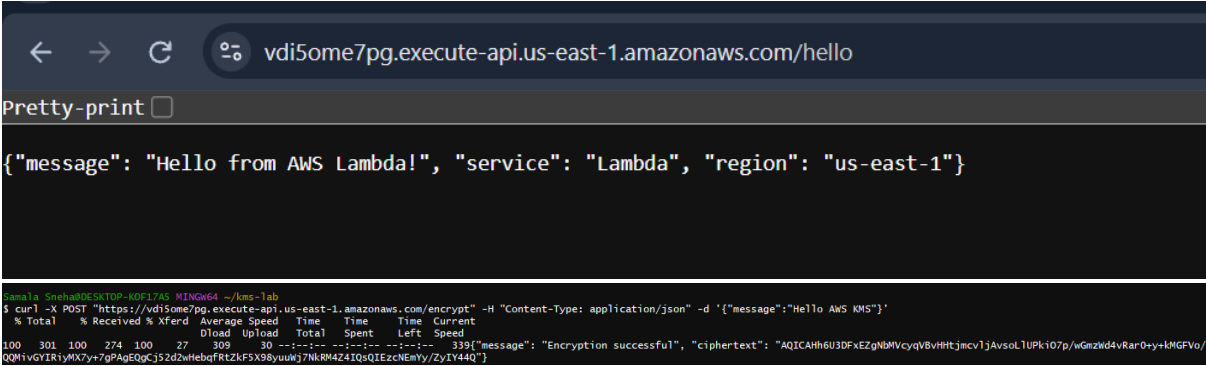
Configure the /encrypt Route

POST /encrypt

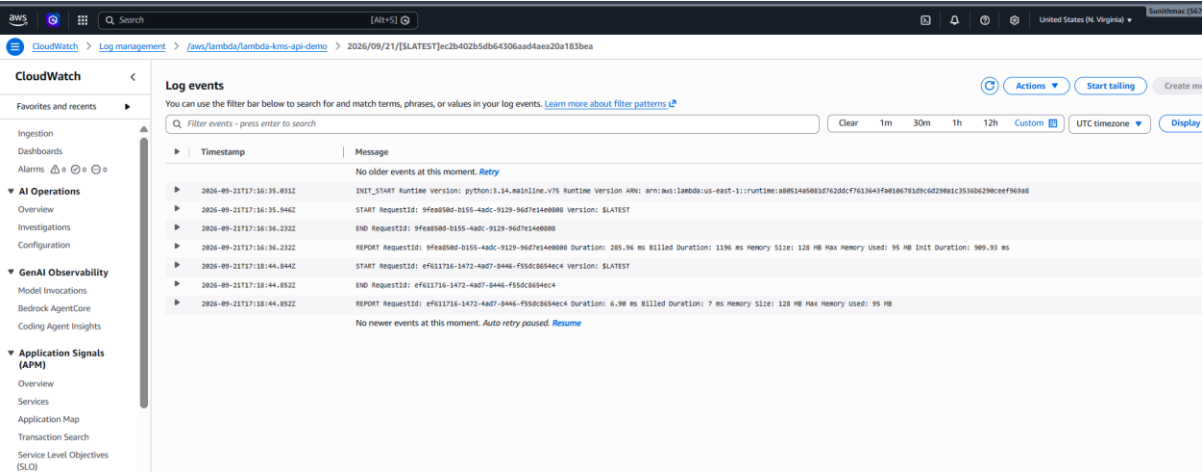
Test the /encrypt Endpoint:

```
"https://YOUR_API_ID.execute-api.us-east-1.amazonaws.com/encrypt" \
-H "Content-Type: application/json" \
-d '{"message": "Hello AWS KMS"}'
```

Implemenatation:



CloudWatch Logs



cloudtrial:

AWS Console → CloudTrail → Event history

The screenshot shows the AWS CloudTrail console. The left sidebar contains navigation links: CloudTrail, Event history, CreateLogStream, Dashboard, Data event coverage, Event history (selected), Insights, Trails, Lake, Dashboards, Query, Event data stores, Integrations, Settings, Pricing, Documentation, Forums, and FAQs. The main content area displays the 'Event record' for the 'CreateLogStream' action. The event details are as follows:

```

1 {
2   "eventVersion": "1.11",
3   "userIdentity": {
4     "type": "AssumedRole",
5     "principalId": "AROAYIIHK02KJUIY57WGT:lambda-kms-api-demo",
6     "arn": "arn:aws:sts:567487919764:assumed-role/lambda-kms-api-demo-role-9r5a8xr6/lambda-kms-api-demo",
7     "accountId": "567487919764",
8     "accessKeyId": "ASIAIYIIHK02KAEQVTH2L",
9     "sessionContext": {
10      "sessionIssuer": {
11        "type": "Role",
12        "principalId": "AROAYIIHK02KJUIY57WGT",
13        "arn": "arn:aws:iam:567487919764:role/service-role/lambda-kms-api-demo-role-9r5a8xr6",
14        "accountId": "567487919764",
15        "userName": "lambda-kms-api-demo-role-9r5a8xr6"
16      },
17      "attributes": {
18        "creationDate": "2026-09-21T17:16:34Z",
19        "mfaAuthenticated": "false"
20      }
21    },
22    "inScopeOf": {
23      "issuerType": "AWS::Lambda::Function",
24      "credentialsIssuedTo": "arn:aws:lambda:us-east-1:567487919764:function:lambda-kms-api-demo",
25      "credentialsIssuedToVersion": "arn:aws:lambda:us-east-1:567487919764:function:lambda-kms-api-demo:$LATEST"
26    }
27  },
28  "eventTime": "2026-09-21T17:16:43Z",
29  "eventSource": "logs.amazonaws.com",
30  "eventName": "CreateLogStream",
31  "awsRegion": "us-east-1",
32  "sourceIPAddress": "3.84.198.18",
33  "userAgent": "aws-lambda-worker/1.0",

```

Conclusion

In this lab, the individual AWS services from the previous labs were combined into a complete serverless encryption application.

API Gateway provides the HTTP interface, while Lambda processes the incoming requests. The Lambda execution role provides controlled access to the customer-managed KMS key.

The /hello endpoint demonstrates basic API Gateway and Lambda integration. The /encrypt endpoint sends a plaintext message to KMS and returns encrypted ciphertext. The /decrypt endpoint sends the ciphertext back to KMS and retrieves the